CHALLENGE SHEET

Nexawallet

An Android wallet app that mints its flag only after a player defeats two independent, in-process gates — a Frida-detection layer that resists direct hooking due to ART inlining, and a hidden, non-exported Activity reachable only from inside the running process.

DIFFICULTY	TRACK	HOST PORT	LAB ID
HARD	BINARY	8090	nexawallet

EASY · MEDIUM · HARD · INSANE

VERIFY THE SOLVE

```
• python3 ../scripts/check.py nexawallet
```

Brief

A custom-built Android crackme (NexaWallet) that simulates a digital wallet with a maximum transfer limit. Defeat an anti-instrumentation layer and a hidden, non-exported Activity, entirely in-process with Frida, to approve a transfer above the limit and mint the flag.

Setup

```
adb install bin/NexaWallet-debug.apk
```

Requires an Android emulator/device, adb, and a Frida stack (see docs/operations.mdx for the tested version matrix). No server-side component — this is a BYOD artifact lab.

Goal

Approve and confirm a transfer above the wallet's limit using dynamic instrumentation, then recover the minted flag. Verify with:

```
python3 check.py "duck\{...\}"
```

What this is

A custom-built Android crackme (**NexaWallet**), designed and built from scratch for this task, then solved end-to-end to validate it and produce the evidence below.

The app simulates a digital wallet with a maximum transfer limit. The player's goal is to get the app to approve and confirm a money transfer **above** that limit, using dynamic instrumentation (Frida) rather than static patching — the app actively detects and blocks common tampering/analysis attempts.

Repo layout

```
.
│
├── lab.yml                        ← openlabs catalog metadata (flag_hash, track,
difficulty)
├── check.py                       ← stdlib-only flag checker (reads flag_hash from lab.
yml)
│
├── bin/                          ← the installable, playable challenge file
│   └── NexaWallet-debug.apk      (placeholder — see bin/README.md)
│
├── docs/                          ← all written evidence/report + maintainer docs,
│   │                                 staged for openlabs (sub-issue #161's "seven-
│   │                                 document pattern" checklist item, plus a
│   │                                 player-facing briefing and the screenshots)
│   ├── README.md
│   ├── player-briefing.mdx       ← player-facing description (no spoilers)
│   ├── challenge-design.mdx
│   ├── official-solution.mdx     ← full spoiler solve, including the narrated,
│   │                                 screenshot-illustrated walkthrough
│   ├── operations.mdx
│   ├── remediation.mdx
│   ├── security-report.mdx
│   ├── validation-report.mdx
│   └── images/                   ← screenshots referenced from official-solution.mdx
│
├── techniques/                    ← technique pages staged for openlabs
│   ├── README.md
│   ├── frida-framework-level-hooking-past-art-inlining.mdx
│   ├── non-exported-activity-is-not-an-access-boundary.mdx
│   ├── in-process-activity-launch-with-frida.mdx
│   └── client-side-authorization-decisions.mdx
│
└── src/                           ← pointer to the full source (private — never ship
to participants)
    ├── BUILD_INSTRUCTIONS.md     ← how to open/build the APK from source
    └── README.md                 ← where the actual Android Studio project lives in
this repo
```

Where to start

If you want to…	Go to
Understand what the challenge is testing	`docs/challenge-design.mdx` (”Skills exercised”)
See the player-facing description (no spoilers)	`docs/player-briefing.mdx`
Reproduce the full solve, step by step, with screenshots	`docs/official-solution.mdx` (”Full narrated walkthrough”)
Understand the security root cause being demonstrated	`docs/security-report.mdx` (”Root cause analysis”)
See mitigation recommendations for real-world systems	`docs/remediation.mdx`
Rebuild the APK from source	`src/BUILD_INSTRUCTIONS.md`
See the catalog-facing challenge design summary	`docs/challenge-design.mdx`
Check a candidate flag	`check.py "duck{...}"`
See what’s verified vs. still pending local confirmation	`docs/validation-report.mdx`

Delivery model (BYOD artifact)

This lab ships as a single installable APK plus documentation — there is no server-side component the solve depends on. The player supplies their own Android emulator/device, adb, and Frida stack (host `frida-tools`

- a matching `frida-server` on the device). See `docs/operations.mdx`

for the full BYOD matrix (tested API levels, decompiler, Frida version pinning).

`docker-compose.yml` and `lab.yml`’s `port: 8090` exist only to satisfy the catalog’s general `docker-compose.yml/validate.py` requirement — they run a distribution-only HTTP server for the APK and docs, and nothing in the intended solve path ever touches that service. See `docs/operations.mdx`, ”Docker role.”

Anti-analysis spec (summary — full table in `docs/challenge-design.mdx`)

Three deterministic instrumentation-detection checks (Frida loopback port, `/proc/self/maps` injected-library scan, live thread-name scan), plus a non-exported hidden Activity and a liveness-checked clearance flag. All four are intentionally, individually bypassable — see `docs/challenge-design.mdx` for which bypasses are in-scope (the intended solve path) versus out-of-scope for this submission.

CI scope

Only static checks (`lab.yml` schema, `check.py` against the known flag/hash pair) can run in CI — the actual solve requires a live emulator/device with Frida attached, which is manual evidence, not automatable here. See `docs/operations.mdx`, ”CI scope.”

Flag design note

The flag is fixed (one value, one `flag_hash`, per catalog requirement) and shipped as an XOR+Base64-obfuscated constant decoded only at `mintClearanceToken()` runtime — not a plaintext string, but also not immune to a sufficiently determined static reader. This trade-off is documented and was an explicit

author decision; see `docs/challenge-design.mdx` ("Why the flag design changed during review") and `docs/validation-report.mdx` ("The accepted static-shortcut trade-off") for the full reasoning.

Design summary

NexaWallet is a single Android app (Kotlin, View Binding, OkHttp). There is no real backend — balances and limits are stored locally for the purposes of this challenge — so the entire security boundary the player must defeat lives **inside the APK itself**, enforced at runtime.

Two independent gates stand between the player and the flag:

1. **Integrity Gate** — a unified, fully deterministic check for active dynamic instrumentation (Frida): a loopback port probe, a scan of the process's own memory map for injected-library signatures, and a scan of live thread names for Frida's own runtime threads. Any one of the three tripping blocks the player with a "Security Check Failed" screen. Hooking the app's own top-level check function directly does **not** work here — see `docs/official-solution.mdx` (full walkthrough) and `docs/security-report.mdx` ("Root cause analysis") for why, and what the player has to pivot to instead.
2. **Review Clearance** — even after Gate 1 is defeated, a transfer above the limit still requires a separate, independent "clearance" flag to be set. That flag is granted only by a screen that is never reachable from any button, menu, or link in the app's normal flow, and is declared non-exported in the manifest, so it can't be opened externally either. The player has to discover it exists by reading the decompiled code/manifest, then trigger it from inside the running process.

Only when **both** gates are satisfied does the app mint and display the flag — nothing is ever hidden as a plain string in the APK.

Evidence status

- x Challenge designed and built (Kotlin/Android, Gradle/AGP, View Binding)
- x Flag redesigned to be fixed/deterministic (catalog requirement), obfuscated, and verified correct by hash/round-trip (see `docs/validation-report.mdx`)
- x `lab.yml`, `check.py`, and all maintainer + player-facing docs (`docs/`), and technique pages (`techniques/`) completed
- x Full walkthrough, objective/skills, player briefing, root-cause and mitigation documentation completed/updated for the current flag design
- x **APK rebuilt from the updated source** by the author, locally (Android SDK isn't available in the authoring environment, so this step had to happen outside it)
- x **Full solve chain re-run end-to-end on the rebuilt APK** — author confirmed the jadx static analysis + Frida dynamic-instrumentation chain in `docs/official-solution.mdx` against the rebuilt APK and recovered the correct flag, matching `lab.yml`'s `flag_hash` (see `docs/validation-report.mdx` for the check, not the flag itself)
- x Release APK SHA-256 recorded in `docs/operations.mdx` (`a80baaf41745df20240a9c65992439cca2457e3e7de4fbd15058d6d791e38fac`)

- [x] **No environment-dependent false positives** — confirmed live on an API 29 emulator: clean device (no Frida) does not block a normal transfer, and re-attaching Frida still blocks as intended (see `docs/validation-report.mdx`)
- [x] `frida-tools/frida-server` versions pinned: `frida-tools 17.22.0` / `frida-server-17.22.0-android-x86` (see `docs/operations.mdx`)

See `docs/validation-report.mdx` for the full sign-off and the two remaining minor open items.